PYTHON ASSIGNMENT

Task-wise Compilation

Tasks 1–12

NAME : PREETHI G
REGISTER NO.: 1924260136

GitHub Repository

https://github.com/preethiofficial94/CSA08-1924260136-Assignment1.git

Contents

1. Task 1 – Understand the Problem and Identify the Data
2. Task 2 – Explore Values and Types in the Python Interpreter
3. Task 3 – Choose Variables and Name the Constants
4. Task 4 – Write Correct Expressions
5. Task 5 – Read Input and Convert Types
6. Task 6 – Organise the Logic into Functions
7. Task 7 – Build a Module
8. Task 8 – Plan the Algorithm / Problem Solving
9. Task 9 – Implement and Test the Calculator
10. Task 10 – Compare Expectations with Execution
11. Task 11 – Recommend and Defend the Final Design
12. Task 12 – Design Challenge

TASK 1- Understand the problem and identify the data

1. Inputs and outputs:

The milk payment system calculates the payments for farmers based on the quantity and quality of milk supplied

Inputs:

***Farmers ID:** An integer or string used to identify each farmer

***Litres:** The quantity of milk supplied,stored as a floating point numbers(float)

***Fat percentage:** The fat content of milk,stored as a *fioat*

***SNF Percentage:** the solid-not-fat content,stored as a *fioat*

***Shift:** A string representating the milk collection shift,such as “morning”, ”evening”

Outputs:

***Rate per litre:** The payment rate calculated based in milk quantity

***Amount**: Total payment for the milk supplied,calculated as *litres*rate_per_litre*

***Ten-day total:** The sum of a farmer's payments over ten days.

2. Constraints:

Constraints are values that do not change during program execution

Examples:

*MIN_FAT=2.0 -Minimum permitted fat percentage

*MAX_FAT=7.0 -Maximum permitted fat percentage

*DAYS=10-Number of days for calculating the total

These contraints improve readability and make the program easier to maintain

3. DATA TYPES

value	Data types	reason
Farmer ID	Int or str	Identifies a farmers
Litres	Float	Milk quantity can contain decimals
Fat percentage	Float	Represents fractional percentages
SNF Percentage	Float	Represents frational percentage
Shift	Str	Stores morning or evening
Rate per litre	Float	Represents the price per litre
Amount	Float	Represents the payment
Ten-day total	Float	Stores accumulated payments
Number of days	int	Represents ten whole days

4. validation rules:

The program must follow these rules:

1. Milk quantity must not be negative

2. Fat percentage must be between 2.0% and 7.0% , inclusive
3. SNF percentage mut be within the permitted range defined by the dairy
4. Farmer ID must not be empty
5. Shift must be either morning or evening
6. The payment must be non-negative
7. Invalid inputs must be rejected before calculating paymemt

These rules help ensure that payments are calculated correctly

5. Assumptions:

* Each farmer has a unique ID

*Milk quantity and quality are recorded for every collection

*The payment rate is determined using the dairy's predefined pricing rues

*The ten days total includes all valid payments recorded for the farmer during ten days

*payment amounts are rounded to two decimal places when displayed

6. Alternative design and complexity analysis :

If each farmer's information is stored In separate variables,such as literal,fat1 and snf1 managing 300 farmers would require many variables.this approach is difficult to aintain and does not scale well.

Instead ,a list or tuple can store each farmers details together ,and a list can store multiple entries.

Time complexity : Processing n entries takes o(n) time because each entry is processed once.

Space complexity: Storing n entries requires (o)n space .

For 300 farmers ,the program can process all entries using a loop and structured data storage . this design is more organized ,reusable and scalable.

Conclusion:

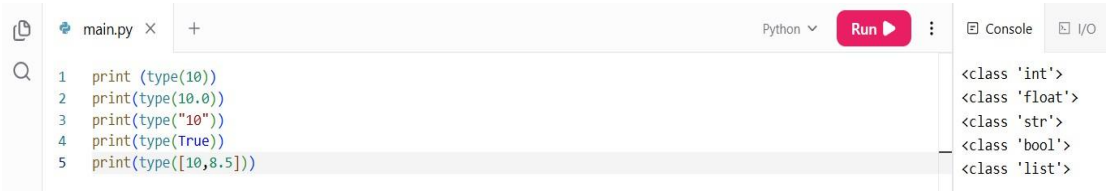
The milk payment system requires clearly defined inputs ,outputs ,constants ,data types, validation rules , and

assumptions . using structured data and loops makes the program easier to maintain and efficient for handling many farmers .

TASK 2 – Explore Values and Types in the Python Interpreter :

1. Identify the Data Types Using type()

Python provides the type() function to identify the data type of a value.



Explanation:

10 – Integer (int), a whole number.

10.0 – Float (float), a decimal number. "10"

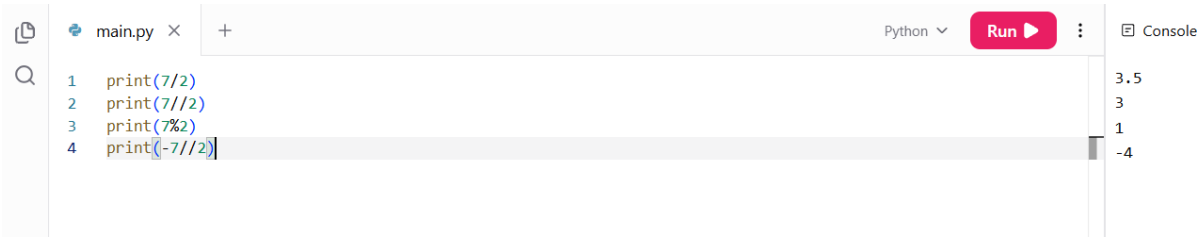
– String (str), text enclosed in quotes.

True – Boolean (bool), represents a logical value. [10,

8.5] – List (list), stores multiple values.

2. Explore Division Operators

Python code:



Explanation:

/ – Performs normal division and returns 3.5.

// – Performs floor division and returns 3.

% – Returns the remainder, which is 1.

-7 // 2 returns -4 because floor division rounds down towards negative infinity.

3. Floating-Point Precision

Python code:



Explanation:

Floating-point numbers cannot represent some decimal values exactly in binary. Therefore, 0.1 + 0.2 produces a value slightly different from 0.3.

4. Handling Money Correctly

Money should not be added directly using floating-point numbers because small precision errors may occur.

Three alternatives are:

1. Integer paise: Store money in paise to avoid floating-point errors.
2. round(): Round values to two decimal places, although this alone does not guarantee exact decimal arithmetic.
3. Decimal: Use Python's decimal module for accurate decimal calculations.

Python code:



Explanation:

*float may produce a small precision error when adding money.

*Decimal provides exact decimal addition when initialized from strings.

*False indicates that floating-point addition does not exactly equal 0.3.

*True indicates that decimal addition gives the exact expected result.

Change Decimal(amount) to Decimal(float(amount))

When a string is converted to float first, its binary approximation is passed to Decimal. This can introduce precision errors, so the total may contain unexpected decimal digits. Therefore, use Decimal(amount) directly.

5. Alternative Concept and Complexity Analysis

Method	**time per addition**	**Exact money calculation?**
Float	**O(1)**	**No**
Integer paise	**O(1)**	**Yes, for amounts represented in whole numbers**
Round()	**O(1)**	**Not guaranteed in every calculation**
Decimal	**Generally O(1) for**	**Yes, when used with**
	fixed size amounts	**appropriate precision**

Space complexity:

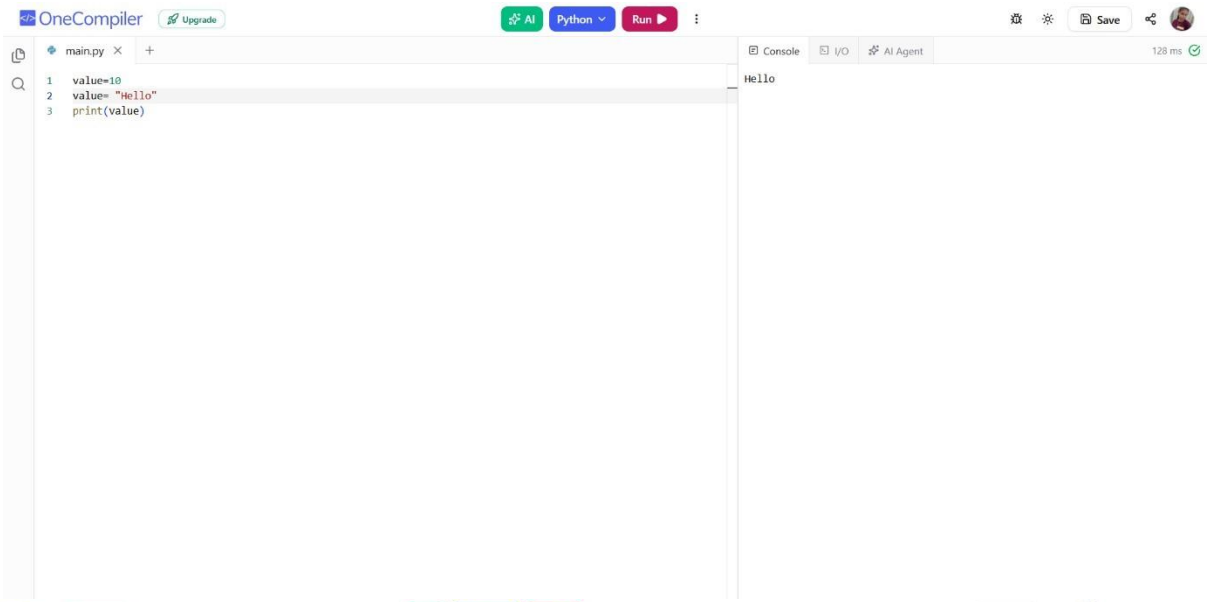
Each stored amount uses a fixed amount of memory for a given numeric representation, so storing n payments requires O(n) space.

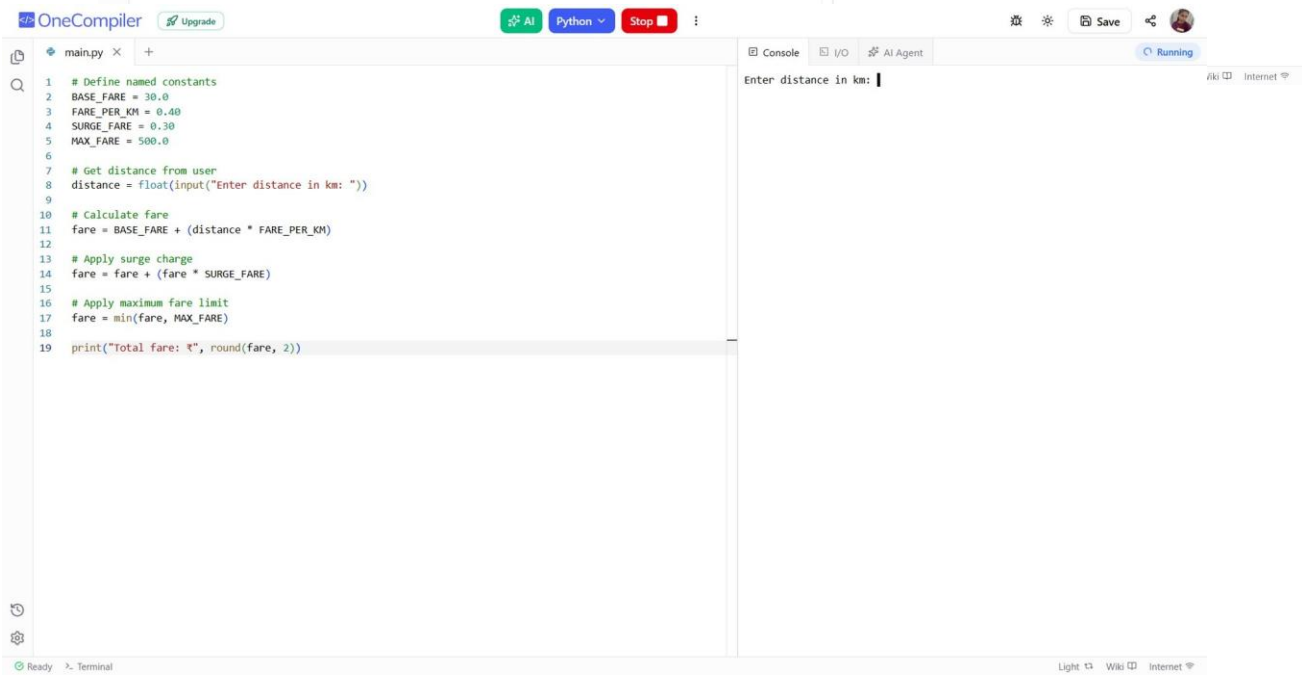
Conclusion:

Python data types and division operators help us understand how values are processed. Floating-point numbers can cause precision errors, so Decimal or integer paise is preferable for accurate financial calculations.

TASK 3 – Choose Variables and Name the Constants (3 Marks)

1. Python Program





2. Naming Rules and Conventions

Variables use lowercase letters, such as distance and fare.

Constants use uppercase letters, such as BASE_FARE and MAX_FARE.

Variable names cannot start with a number or contain spaces.

Meaningful names make programs easier to understand.

3. Why Python Has No True Constants

Python does not strictly prevent constants from being changed. Uppercase names are used by convention to indicate values that should not change.

Example: BASE_FARE = 30.0

4. Dynamic Typing

Python allows a variable to store different data types.

TASK 4 – Write Correct Expressions

1. Python Expressions



2. **Operator Precedence and Associativity**

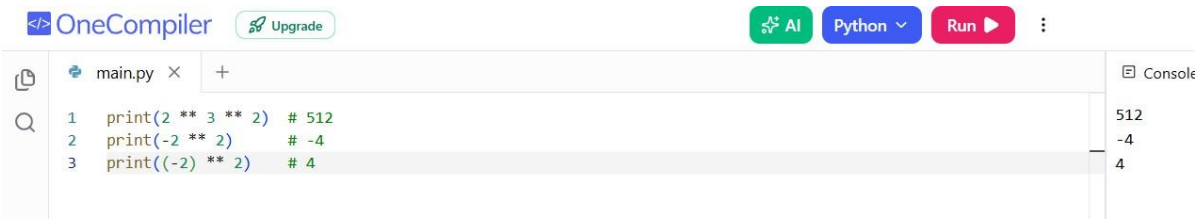
** performs exponentiation before multiplication and addition. Unary minus affects the sign of a value.

Parentheses are evaluated first.

Multiplication and division have higher precedence than addition and subtraction.

Operators with the same precedence are generally evaluated from left to right, except exponentiation, which associates from right to left.

Example:



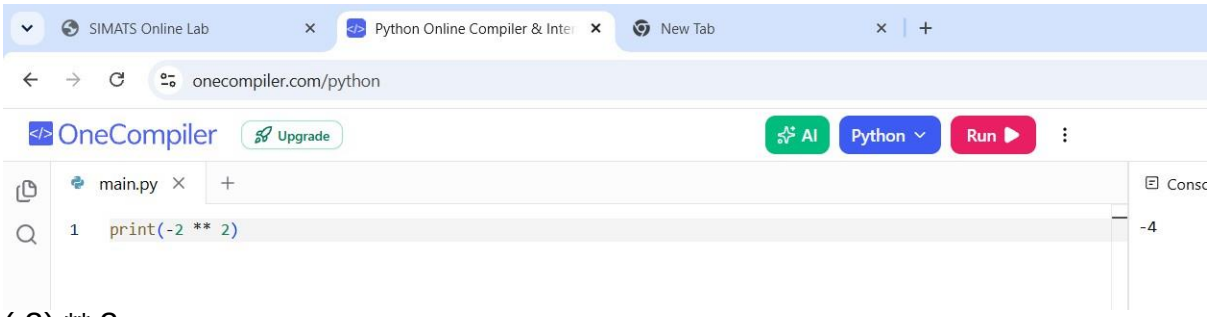
3. Three Unexpected Result

Example 1: Floating-point precision



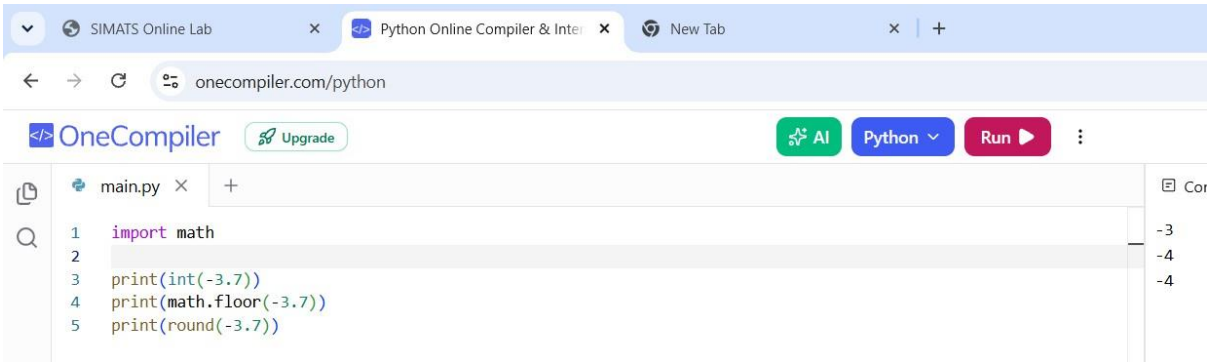
Explanation: Floating-point numbers cannot represent every decimal value exactly. Using round() gives the intended step count of 7.

Example 2: Unary minus



(-2) ** 2.

Example 3: Integer conversion

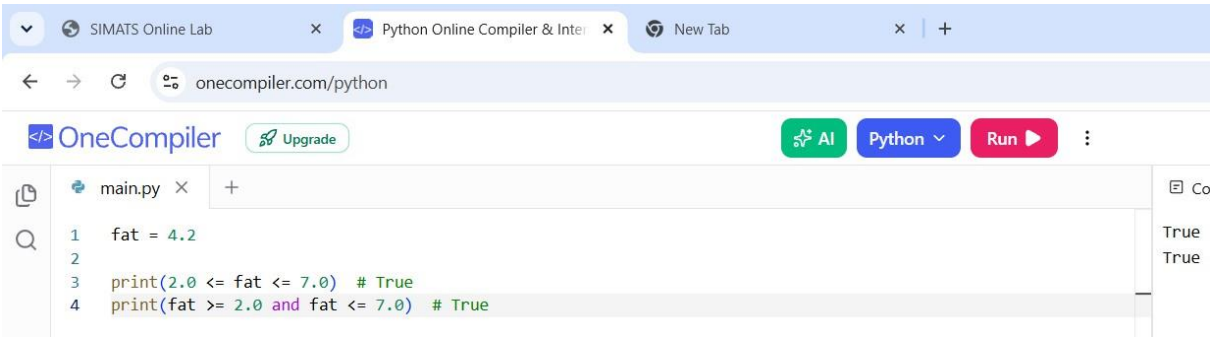


Explanation:

int() truncates towards zero.

math.floor() rounds down to the next lower integer. round() rounds to the nearest integer.

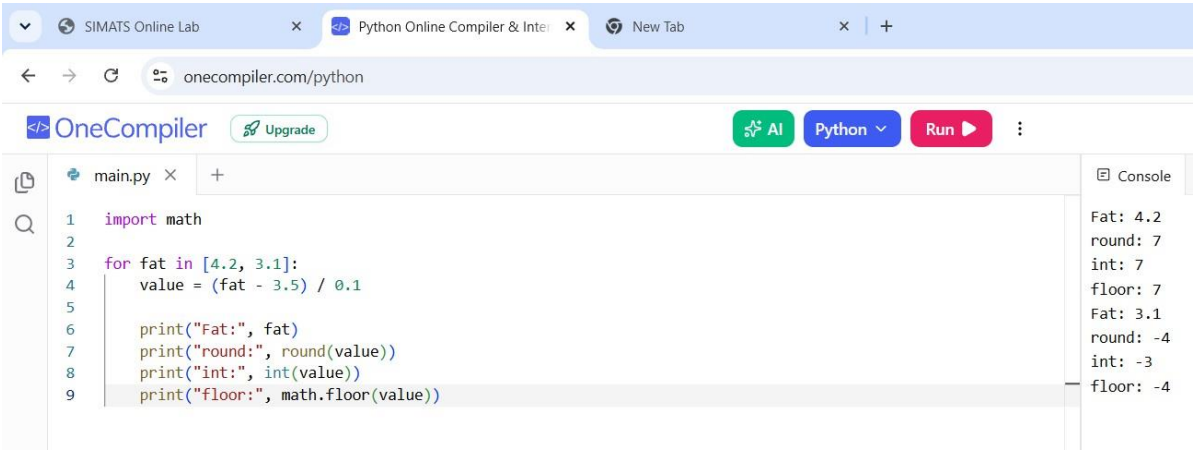
4. Comparison Chaining and Boolean Operators



Explanation: Comparison chaining checks both conditions. Boolean operators such as and, or, and not combine or reverse logical conditions.

5. Alternative Concept and Complexity Analysis

For fat values 4.2 and 3.1, calculate the steps using round(), int(), and math.floor().



Expected results(step counts):

fat	Calculated value	Round()	Int()	Math.floor()
4.2	Approx. 7	7	7	7
3.1	Approx. -4	-4	-3	-4

Explanation:

round() returns the nearest integer.

int() truncates towards zero.

math.floor() returns the greatest integer less than or equal to the value.

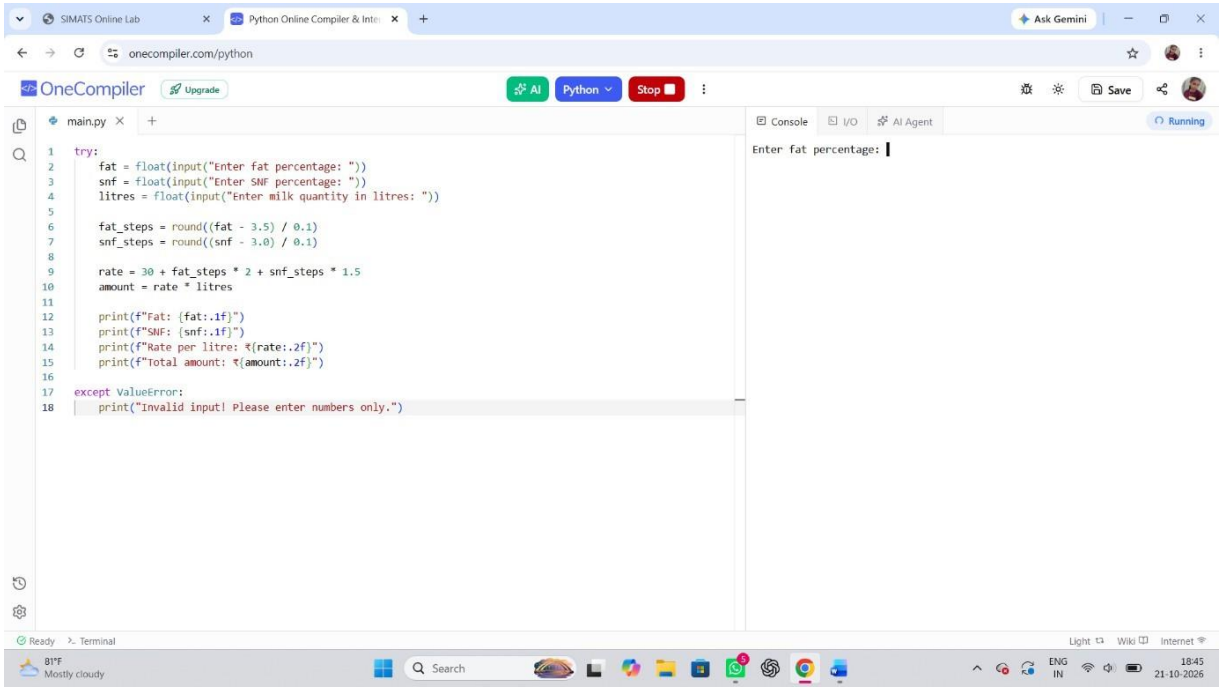
Complexity:

Each operation takes constant time, O(1), and constant extra space, O(1).

Conclusion: Correct expressions require understanding operator precedence, floating-point precision, type conversion, and comparison chaining. Using round() can help calculate step counts when the values are affected by floating-point precision.

Task 5 – Read input and convert types

1. Python program:




```
try:
    fat = float(input("Enter fat percentage: "))
    snf = float(input("Enter SNF percentage: "))
    litres = float(input("Enter milk quantity in litres: "))

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 3.0) / 0.1)

    rate = 30 + fat_steps * 2 + snf_steps * 1.5
    amount = rate * litres

    print(f"Fat: {fat:.1f}")
    print(f"SNF: {snf:.1f}")
    print(f"Rate per litre: ₹{rate:.2f}")
    print(f"Total amount: ₹{amount:.2f}")

except ValueError:
    print("Invalid input! Please enter numbers only.")
```

2. Explanation

Why does input() return a string?

The input() function reads the user's entry as text. We use float() to convert decimal values and int() to convert whole numbers.

What happens with invalid input?

8.5 — valid decimal input.

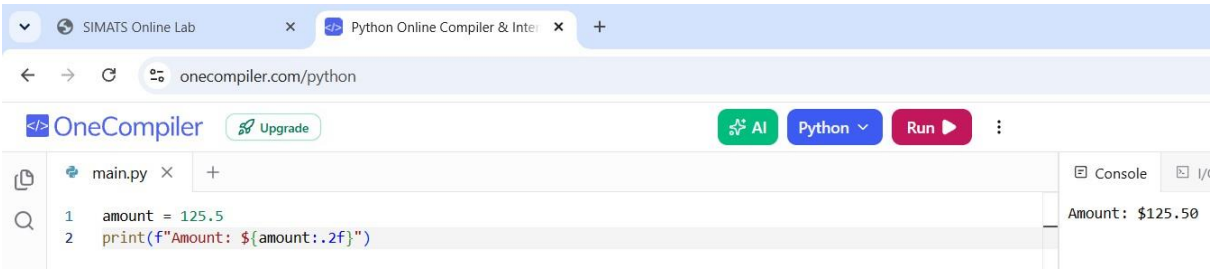
8,5 — invalid for Python's float() function. eight

— invalid because it is not a number.

The try-except block catches invalid input and displays an error message instead of stopping the program.

How do we format the output?

For example:



Here, .2f displays two decimal places.

3. Alternative Concept and Complexity Analysis:

An alternative is to convert input directly using float() inside a try-except block.

String checking: Checks the input before conversion, but some valid numeric formats may be rejected.

Try-except: Attempts conversion and catches invalid values such as "eight" and "8,5".

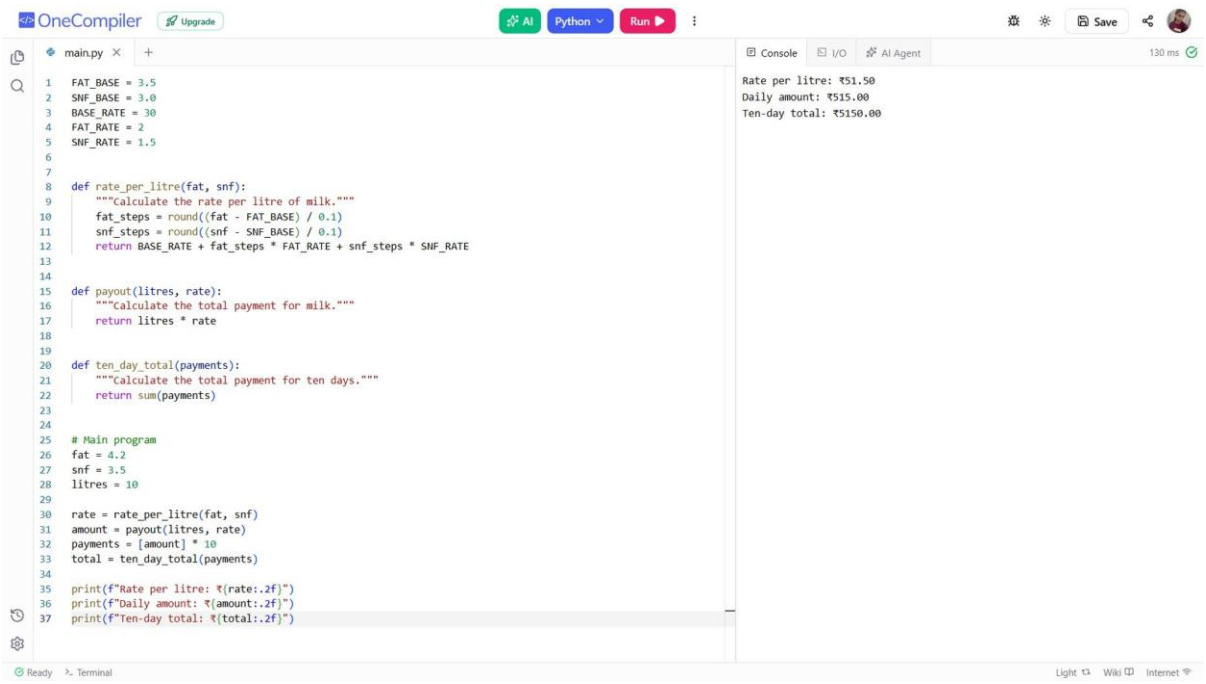
Time complexity: O(1) per entry for both approaches, assuming ordinary input lengths.

Space complexity: O(1) extra space for a fixed number of entries.

Conclusion: Using float() with try-except is a simple and reliable way to convert input, handle invalid values, and display neatly formatted results.

Task 6- Organise the logic into functions

1. Python program:




```python
FAT_BASE = 3.5
SNF_BASE = 3.0
BASE_RATE = 30
FAT_RATE = 2
SNF_RATE = 1.5


def rate_per_litre(fat, snf):
    """Calculate the rate per litre of milk."""
    fat_steps = round((fat - FAT_BASE) / 0.1)
    snf_steps = round((snf - SNF_BASE) / 0.1)
    return BASE_RATE + fat_steps * FAT_RATE + snf_steps * SNF_RATE


def payout(litres, rate):
    """Calculate the total payment for milk."""
    return litres * rate


def ten_day_total(payments):
    """Calculate the total payment for ten days."""
    return sum(payments)


# Main program
fat = 4.2
snf = 3.5
litres = 10

rate = rate_per_litre(fat, snf)
amount = payout(litres, rate)
payments = [amount] * 10
total = ten_day_total(payments)

print(f"Rate per litre: ₹{rate:.2f}")
print(f"Daily amount: ₹{amount:.2f}")
print(f"Ten-day total: ₹{total:.2f}")
```

2. Explanation:

1. **Functions needed:**

rate_per_litre() calculates the milk rate. payout()

calculates the payment for the milk.

ten_day_total() calculates the total payment for ten days.

2. **Returning a value vs. printing:** return

sends a result back to the calling code. print()

displays the result on the screen.

3. Local and global variables:

Global constants, such as BASE_RATE, can be read inside functions.

Local variables exist only inside their function.

Constants should not be modified inside functions.

3. Docstrings Docstrings describe what each function does. They are written using triple quotation marks ("""..."""").

4. **Alternative Concept and Complexity Analysis:**

Without functions: Repeated code makes the program difficult to maintain.

With functions: Code becomes reusable, readable, and easier to test.

Lambda expressions: Suitable for short expressions but less suitable for organising several calculations.

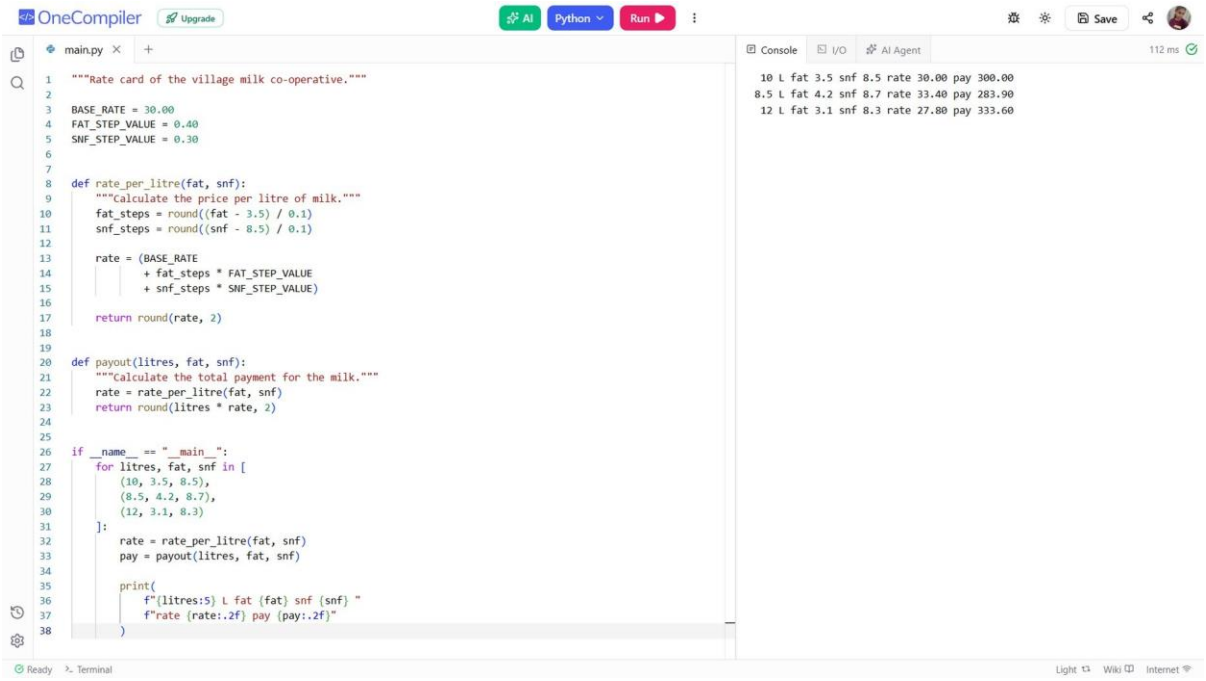
Time complexity: Calculating one entry takes O(1). Processing n entries takes O(n).

Space complexity: O(1) for one entry, excluding stored payment history.

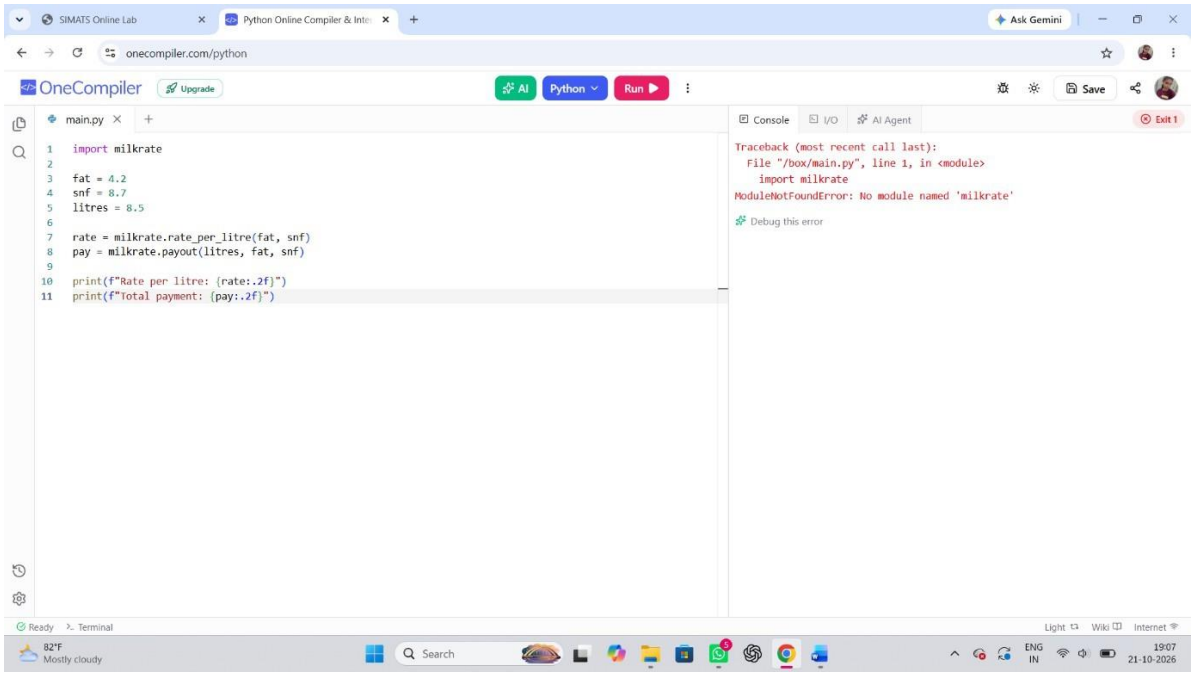
Conclusion: Functions organise the calculation into reusable parts, reduce repeated code, and improve readability and maintenance.

TASK – 7 Build a module

1. File 1:



2. Expected output:



4. Explanation:

1. **Importing a module:**

import milkrate imports the complete module.

from milkrate import payout imports only the payout() function. import

milkrate as mr imports the module using the alias mr.

2. Purpose of if __name__== "__main__":

This condition runs the demonstration code only when milkrate.py is executed directly. When another script imports the module, the demonstration table is not printed.

3. Standard modules

Python provides built-in modules such as math for mathematical operations and decimal for decimal arithmetic.

4. Module search path

Python searches for modules in locations listed in sys.path, including the current script's directory and installed library locations.

5. Alternative Concept and Complexity Analysis

Copying code: Every script needs updating when the milk rate changes.

Using a module: Update the rate card in one place and reuse it in other scripts.

Import overhead: Python loads the module the first time it is imported in a process and caches it in sys.modules.

Time complexity: Each rate or payment calculation takes O(1).

Space complexity: The calculations use O(1) extra space.

Conclusion: Modules improve code reuse, reduce duplication, and make programs easier to maintain. The if _name_____== "_main_": condition prevents the demonstration table from appearing when the module is imported**.**

TASK – 8 Plan the Algorithm problem solving

1. Steps of Algorithmic Problem Solving:

Understand: Identify the milk quantity, fat percentage, SNF percentage, and rate.

Plan: Calculate the rate per litre and payment for each entry. Write:

Convert the pseudocode into a Python program.

Test: Test the program using three entries and verify the results.

OneCompiler Upgrade

main.py

```
BEGIN
    SET ten_day_total = 0

    FOR each of 10 days:
        READ litres, fat, snf

        IF input is invalid:
            DISPLAY error message
            CONTINUE

        fat_steps = ROUND((fat - 3.5) / 0.1)
        snf_steps = ROUND((snf - 8.5) / 0.1)

        rate = 30 + (fat_steps * 0.40)
                  + (snf_steps * 0.30)

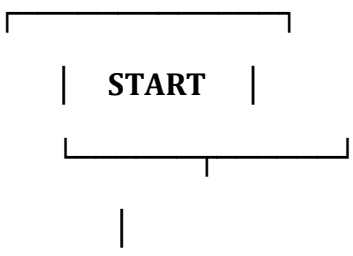
        payment = ROUND(litres * rate, 2)

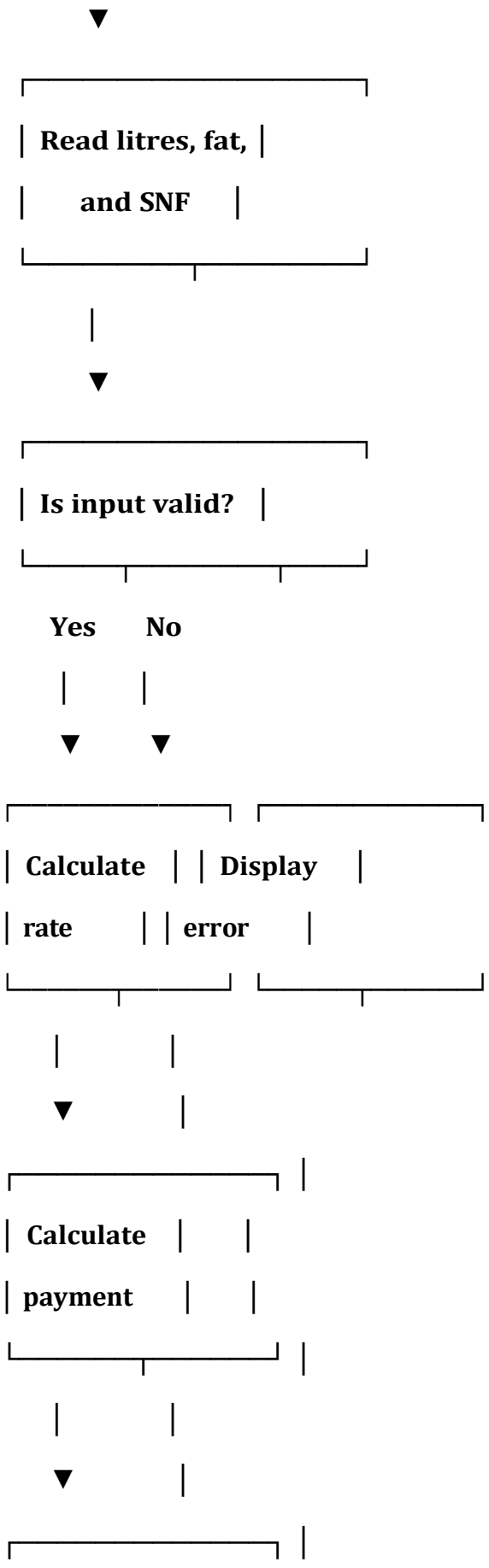
        ten_day_total = ten_day_total + payment

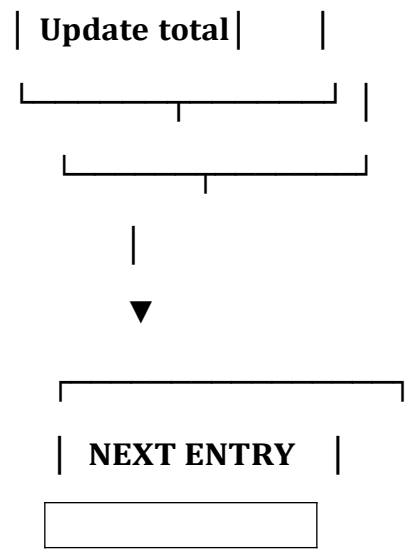
        DISPLAY rate and payment

    DISPLAY ten_day_total
END
```

2. **Flow chart for validating one entry:**




Read litres, fat, and SNF
Is input valid?
Yes
No
Calculate rate
Display error
Calculate payment



4. Hand Trace for Three Entries

Base rate = ₹30.00

Fat step value = ₹0.40

SNF step value = ₹0.30

Entry	litres	fat	SNF	Rate/litre	Payment
1	10	3.5	8.5	$30.00	$300.00
2	8.5	4.2	8.7	$33.40	$283.90
3	12	3.1	8.3	$27.80	$333.60

Total:
$917.50

Calculation for Entry 2:

Fat steps = (4.2 − 3.5) / 0.1 ≈ 7

SNF steps = (8.7 − 8.5) / 0.1 ≈ 2

Rate = 30 + (7 × 0.40) + (2 × 0.30) = ₹33.40

Payment = 8.5 × 33.40 = ₹283.90

5. Alternative Concept and Complexity Analysis:

There are two methods to calculate the ten-day total.

Method 1: Store payments in a list

Store every daily payment and add them at the end. Time

complexity: O(n).

Space complexity: O(n).

Suitable when individual payment records are required later.

Method 2: Running total

Add each payment to the total immediately. Time

complexity: O(n).

Extra space complexity: O(1).

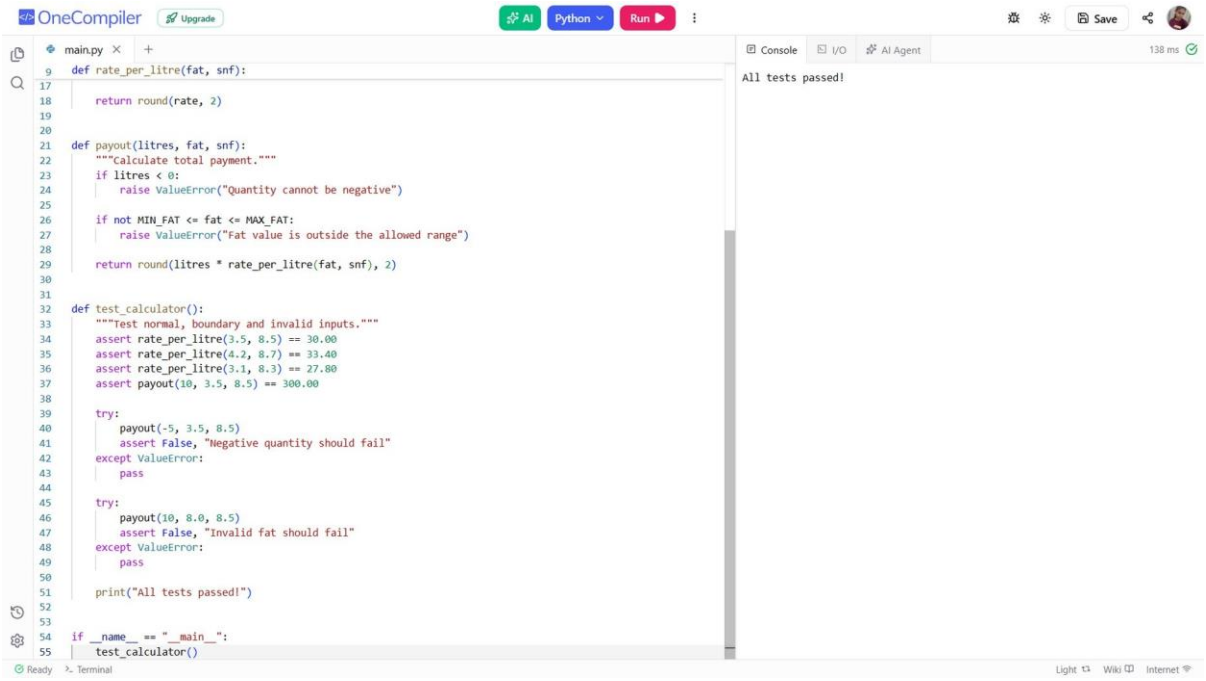
Suitable when only the final total is required.

Conclusion:

Algorithmic problem solving helps organise the program into clear steps before coding. Using a running total saves memory, while storing payments is useful when individual records are needed. Testing with three entries helps verify the calculations before processing all ten days.

TASK – 9 Implement and test the calculator

1. **Complete the program :**




```python
def rate_per_litre(fat, snf):

    return round(rate, 2)


def payout(litres, fat, snf):
    """Calculate total payment."""
    if litres < 0:
        raise ValueError("Quantity cannot be negative")

    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat value is outside the allowed range")

    return round(litres * rate_per_litre(fat, snf), 2)


def test_calculator():
    """Test normal, boundary and invalid inputs."""
    assert rate_per_litre(3.5, 8.5) == 30.00
    assert rate_per_litre(4.2, 8.7) == 33.40
    assert rate_per_litre(3.1, 8.3) == 27.80
    assert payout(10, 3.5, 8.5) == 300.00

    try:
        payout(-5, 3.5, 8.5)
        assert False, "Negative quantity should fail"
    except ValueError:
        pass

    try:
        payout(10, 8.0, 8.5)
        assert False, "Invalid fat should fail"
    except ValueError:
        pass

    print("All tests passed!")


if __name__ == "__main__":
    test_calculator()
```

2 .Test table:

Test case	Input(litres,fat,SNF)	EXPECTED RESULT	Actual result
Normal	10,3.5,8.5	$300.00	$300.00
Fat above standard	8.5,4.2,8.7	$283.90	$283.90
Fat below standard	12,3.1,8.3	$333.60	$333.60
Negative quality	-5,3.5,8.5	error	error
Invalid fat	10,8.0,8.5	Error	Error

3. five entries – hand calculation:

Entry	litres	fat	SNF	payment
1	10	3.5	8.5	$300.00
2	8.5	4.2	8.7	$283.90
3	12	3.1	8.3	$333.60
4	5	8.5	8.5	$150.00

5	**6**	**8.5**	**8.5**	**$312.00**
Total:				**$1379.50**

4. Alternative Concept and Complexity Analysis

Manual testing:

The user enters every test case manually.

It takes more time to repeat tests after code changes.

There is a possibility of entering incorrect test values.

Automated testing using assert:

Tests run automatically.

Errors are identified quickly.

Tests can be repeated after every code change.

Time complexity: O(n) for testing n cases.

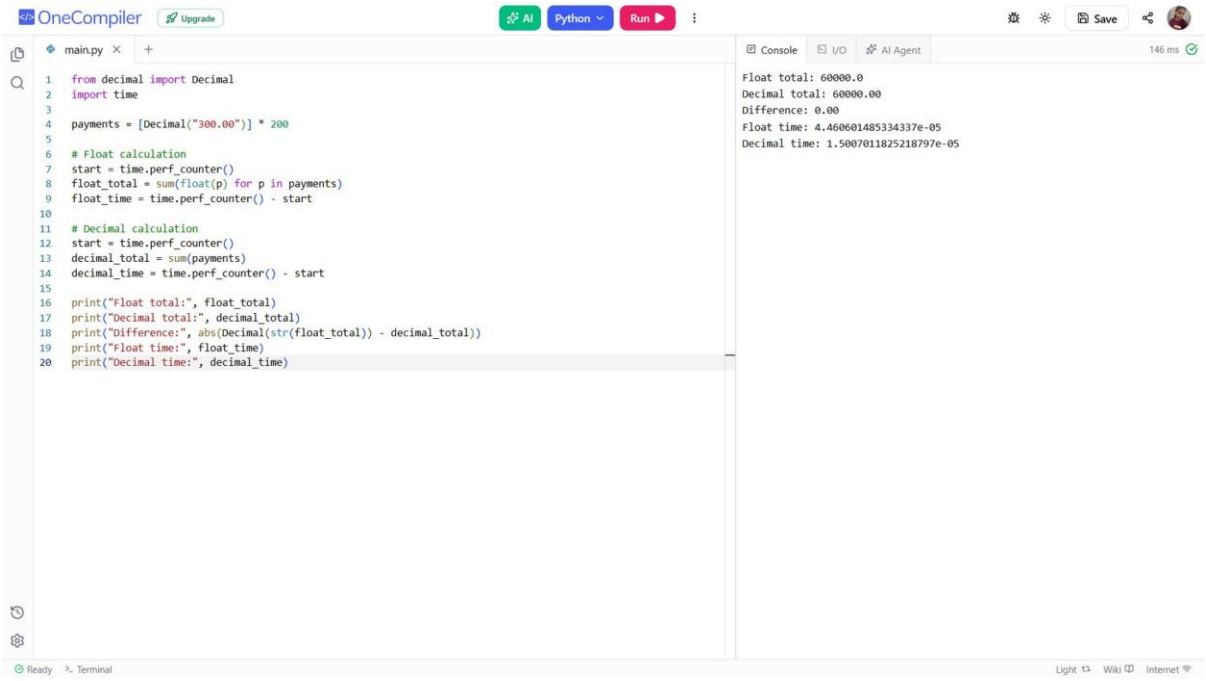
Space complexity: O(1) extra space when each test is processed individually.

Conclusion: Automated testing makes the milk calculator easier to verify, repeat, and maintain. Boundary and invalid-input tests help identify calculation and validation errors.

Important: This program validates negative quantities and fat values outside the allowed range. If your assignment specifies a different permitted fat range or SNF range, adjust the validation accordingly.

TASK – 10 Compare expectations with execution:

1. Python program:



2. Expected and Observed Results

The program can be tested with 200, 6,000, and 150,000 payments.

Number of payments

Expected total

Float result

Decimal result 200

₹60,000.00

Approximately ₹60,000

₹60,000.00

6,000

₹1,800,000.00

Approximately ₹1,800,000

₹1,800,000.00

150,000

₹45,000,000.00

Approximately ₹45,000,000

₹45,000,000.00

Note: These are expected values, not measured execution results. Run the program to obtain actual timings and differences.

2. **Explanation:**

Float vs Decimal

float stores numbers in binary floating-point format, which can introduce small rounding errors.

Decimal represents decimal values exactly when initialized from strings, making it useful for money calculations.

Rounding payments

Rounding each payment to two decimal places before adding gives the total of individual payments.

Rounding only the final total can produce a different result.

For financial calculations, the appropriate method depends on the required payment and accounting rules.

Execution time

time.perf_counter() measures the time taken by each calculation.

Decimal operations are generally slower than float operations.

4. Alternative Concept and Complexity Analysis

A rate table can store previously calculated rates for each (fat, snf) pair.

Recalculation: Calculate the rate for every entry; typically O(1) per entry.

Dictionary lookup: Retrieve a previously calculated rate in average O(1) time.

Extra space: O(k), where k is the number of distinct (fat, snf) pairs stored.

A rate table is useful when the same pairs occur repeatedly. If every pair is different, storing all rates may use extra memory without much benefit.

Conclusion

Float calculations are fast but may introduce small precision errors. Decimal calculations are more suitable for financial values requiring exact decimal representation. Timing measurements and comparison of totals help determine the most appropriate method for the milk payment calculator.

TASK 11 – Recommend and defend the final design

1. Recommend final design:

I recommend using Python functions, a separate rate-card module, and the Decimal type for calculating milk payments. This design improves accuracy, readability, and maintenance.

2. Data Types and Rounding Rules:

Use Decimal for money calculations to avoid floating-point precision errors.

Round each farmer's payment to two decimal places using the agreed accounting rule.

Store the rate and payment with two decimal places.

3. Rate Revision:

Keep the rate card in a separate configuration file, such as JSON.

When rates change, update the file instead of modifying the main program.

Validate the new rates before using them.

4. Limitations:

Incorrect input can still cause errors if validation is incomplete.

Decimal calculations may be slower than float calculations.

The program requires backups and access controls to protect payment records.

5. District-Union Scale:

The design can support a district-level milk union if it is extended with a database, user authentication, transaction records, backups, and proper testing. A simple standalone script is not sufficient for a large organisation.

6. Alternative Concept and Complexity Analysis:

The rate card can be stored in a text or JSON file instead of being written directly in the Python module.

Advantages:

Rates can be changed without editing the Python code. The

same rate card can be shared by multiple programs.

Configuration is easier to maintain.

Disadvantages:

The program needs additional time to read and validate the file at startup.

The file occupies additional storage space.

Invalid or missing files can prevent the program from starting.

Complexity:

Reading a rate card with t values takes O(t) time and O(t) space.

A dictionary lookup takes average O(1) time.

Calculating one payment takes O(1) time.

7. Conclusion:

The recommended design uses Decimal for accurate financial calculations, functions for reusable logic, and a separate JSON rate card for easy updates. Testing, input validation, backups, and access controls are essential for reliable operation. The design can be extended to district-union scale with suitable database and security improvements.

TASK – 12 DESIGN CHALLENGE

A. Impact Analysis – 2 Marks

The new requirements introduce three changes:

Base rate: Changed from ₹30.00 to ₹32.00 per litre.

Fat step value: Changed from ₹0.40 to ₹0.45 for every 0.1% fat difference. Seasonal

rate: Different rates may apply from April to June.

Bonus: Farmers supplying more than 500 litres in ten days receive an additional ₹0.50 per litre.

The constants, rate calculation function, and payment function must be updated. Since fat and SNF readings contain two decimal places, the program must round the step counts to the nearest whole step.

B. Evaluation of the Original Design – 2 Marks

The original design is reusable, but its constants and payment calculation need modification.

Example:

Base rate = ₹32.00

Fat = 4.20%

SNF = 8.70%

Fat step value = ₹0.45

SNF step value = ₹0.30

Fat steps = round((4.20 − 3.50) / 0.10) = 7

SNF steps = round((8.70 − 8.50) / 0.10) = 2

Rate = 32.00 + (7 × 0.45) + (2 × 0.30)

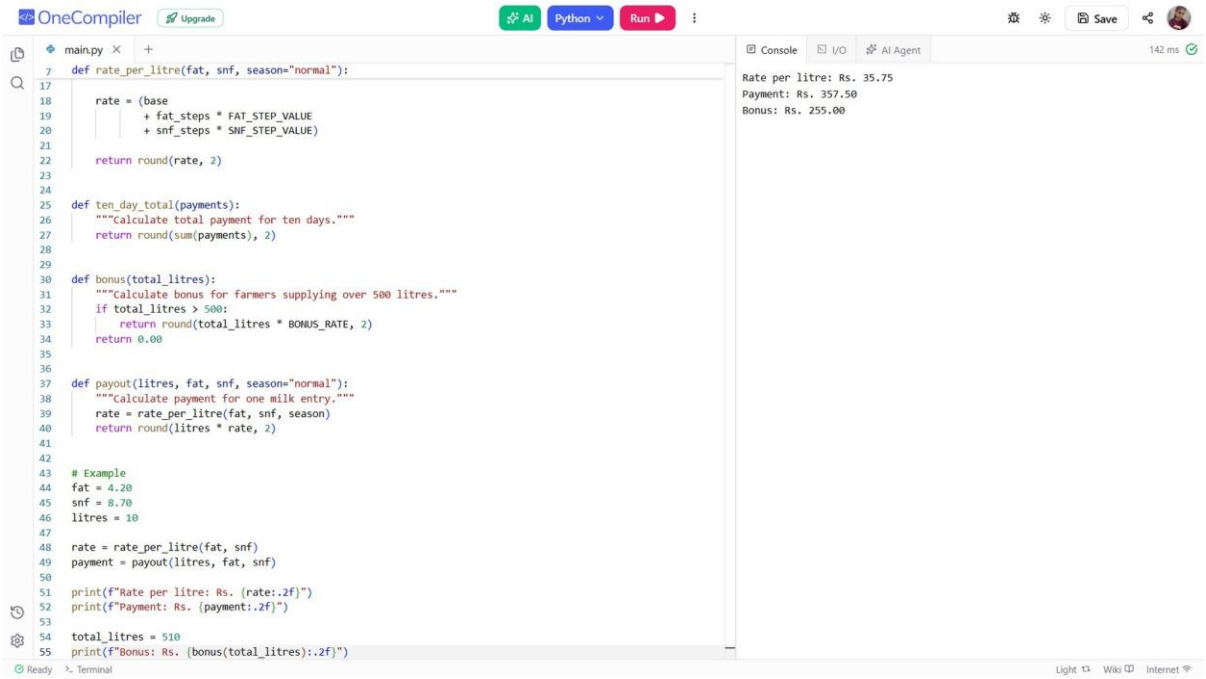
Rate = ₹35.75 per litre

Payment for 10 litres = 10 × ₹35.75 = ₹357.50

Therefore, the original design can be retained with modifications to its constants and functions.

C. Adaptation or Redesign – 2 Marks

Revised Python Program



D. Evidence-Based Defence – 2 Marks

Test case

Expected result

Normal season, 10 litres, fat 4.20, SNF 8.70

₹357.50

Summer season, same readings

₹367.50

Total supply of 500 litres

₹0 bonus

Total supply of 510 litres

₹255.00 bonus

Advantages

Functions make the program reusable.

Constants make rate revisions easier.

Seasonal rates can be selected through a parameter. The

bonus is calculated separately.

Limitations

The actual seasonal rate must be confirmed. Input

validation is needed for invalid readings.

Payment records should be stored securely for auditing.

Conclusion

I recommend modifying the original design rather than completely redesigning it. Updating the constants, adding seasonal rate selection, and implementing a separate bonus function will make the calculator easier to maintain and extend. The final implementation should be tested against the cooperative's official rate card and bonus rules before deployment